

Language Design Criteria

2.1	Historical Overview	27
2.2	Efficiency	28
2.3	Regularity	30
2.4	Security	33
2.5	Extensibility	34
2.6	C++: An Object-Oriented Extension of C	35
2.7	Python: A General-Purpose Scripting Language	38

CHAPTER 2

What is good programming language design? By what criteria do we judge it? Chapter 1 emphasized human readability and mechanisms for abstraction and complexity control as key requirements for a modern programming language. Judging a language by these criteria is difficult, however, because the success or failure of a language often depends on complex interactions among many language mechanisms. Defining the “success” or “failure” of a programming language is also complex; for now, let’s say that a language is successful if it satisfies any or all of the following criteria:

1. Achieves the goals of its designers
2. Attains widespread use in an application area
3. Serves as a model for other languages that are themselves successful

Practical matters not directly connected to language definition also have a major effect on the success or failure of a language. These include the availability, price, and quality of translators. Politics, geography, timing, and markets also have an effect. The C programming language has been a success at least partially because of the success of the UNIX operating system, which supported its use. COBOL, though chiefly ignored by the computer science community, continues as a significant language because of its use in industry, and because of the large number of legacy applications (old applications that continue to be maintained). The language Ada achieved immediate influence because of its required use in certain U.S. Defense Department projects. Java and Python have achieved importance through the growth of the Internet and the free distribution of these languages and their programming environments. The Smalltalk language never came into widespread use, but most successful object-oriented languages borrowed a large number of features from it.

Languages succeed for as many different reasons as they fail. Some language designers argue that an individual or small group of individuals have a better chance of creating a successful language because they can impose a uniform design concept. This was true, for example, with Pascal, C, C++, APL, SNOBOL, and LISP, but languages designed by committees, such as COBOL, Algol, and Ada, have also been successful.

When creating a new language, it’s essential to decide on an overall goal for the language, and then keep that goal in mind throughout the entire design process. This is particularly important for special-purpose languages, such as database languages, graphics languages, and real-time languages, because the particular abstractions for the target application area must be built into the language design. However, it is true for general-purpose languages as well. For example, the designers of FORTRAN focused on efficient execution, whereas the designers of COBOL set out to provide an English-like nontechnical readability. Algol60 was designed to provide a block-structured language for describing algorithms and

Pascal was designed to provide a simple instructional language to promote top-down design. Finally, the designer of C++ focused on the users' needs for greater abstraction while preserving efficiency and compatibility with C.

Nevertheless, it is still extremely difficult to describe good programming language design. Even noted computer scientists and successful language designers offer conflicting advice. Niklaus Wirth, the designer of Pascal, advises that simplicity is paramount (Wirth [1974]). C. A. R. Hoare, a prominent computer scientist and co-designer of a number of languages, emphasizes the design of individual language constructs (Hoare [1973]). Bjarne Stroustrup, the designer of C++, notes that a language cannot be merely a collection of “neat” features (Stroustrup [1994], page 7). Fred Brooks, a computer science pioneer, maintains that language design is similar to any other design problem, such as designing a building (Brooks [1996]).

In this chapter, we introduce some general design criteria and present a set of more detailed principles as potential aids to the language designer and ultimately the language user. We also give some specific examples to emphasize possible good and bad choices, with the understanding that there often is no general agreement on these issues.

2.1 Historical Overview

In the early days of programming, machines were extremely slow and memory was scarce. Program speed and memory usage were, therefore, the prime concerns. Also, some programmers still did not trust compilers to produce efficient executable code (code that required the fewest number of machine instructions and the smallest amount of memory). Thus, one principal design criterion really mattered: **efficiency of execution**. For example, FORTRAN was specifically designed to allow the programmer to generate compact code that executed quickly. Indeed, with the exception of algebraic expressions, early FORTRAN code more or less directly mapped to machine code, thus minimizing the amount of translation that the compiler would have to perform. Judging by today's standards, creating a high-level programming language that required the programmer to write code nearly as complicated as machine code might seem counterproductive. After all, the whole point of a high-level programming language is to make life easier for the programmer. In the early days of programming, however, **writability**—the quality of a language that enables a programmer to use it to express a computation clearly, correctly, concisely, and quickly—was always subservient to efficiency. Moreover, at the time that FORTRAN was developed, programmers were less concerned about creating programs that were easy for people to read and write, because programs at that time tended to be short, written by one or a few programmers, and rarely revised or updated except by their creators.

By the time COBOL and Algol60 came on the scene, in the 1960s, languages were judged by other criteria than simply the efficiency of the compiled code. For example, Algol60 was designed to be suitable for expressing algorithms in a logically clear and concise way—in other words, unlike FORTRAN, it was designed for easy reading and writing by people. To achieve this design goal, Algol60's designers incorporated block structure, structured control statements, a more structured array type, and recursion. These features of the language were very effective. For example, C. A. R. Hoare understood how to express his QUICKSORT algorithm clearly only after learning Algol60.

COBOL's designers attempted to improve the readability of programs by trying to make them look like ordinary written English. In fact, the designers did not achieve their goal. Readers were not able to

easily understand the logic or behavior of COBOL programs. They tended to be so long and verbose that they were harder to read than programs written in more formalized code. But human readability was, perhaps for the first time, a clearly stated design goal.

In the 1970s and early 1980s, language designers placed a greater emphasis on simplicity and abstraction, as exhibited by Pascal, C, Euclid, CLU, Modula-2, and Ada. Reliability also became an important design goal. To make their languages more reliable, designers introduced mathematical definitions for language constructs and added mechanisms to allow a translator to partially prove the correctness of a program as it performed the translation. However, such program verification systems had limited success, primarily because they necessitated a much more complex language design and translator, and made programming in the language more difficult than it would be otherwise. However, these efforts did lead to one important related development, strong data typing, which has since become standard in most languages.

In the 1980s and 1990s, language designers continued to strive for logical or mathematical precision. In fact, some attempted to make logic into a programming language itself. Interest in functional languages has also been rekindled with the development of ML and Haskell and the continued popularity of Lisp/Scheme.

However, the most influential design criterion of the last 25 years has come from the object-oriented approach to abstraction. As the popularity of the object-oriented languages C++, Java, and Python soared, language designers became ever more focused on abstraction mechanisms that support the modeling of real-world objects, the use of libraries to extend language mechanisms to accomplish specific tasks, and the use of object-oriented techniques to increase the flexibility and reuse of existing code.

Thus, we see that design goals have changed through the years, as a response both to experience with previous language designs and to the changing nature of the problems addressed by computer science. Still, readability, abstraction, and complexity control remain central to nearly every design decision.

Despite the importance of readability, programmers still want their code to be efficient. Today's programs process enormous data objects (think movies and Web searches) and must run on miniature computers (think smart phones and tablets). In the next section, we explore the continuing relevance of this criterion to language design.

2.2 Efficiency

Language designers nearly always claim that their new languages support efficient programs, but what does that really mean? Language designers usually think of the efficiency of the target code first. That is, they strive for a language design that allows a translator to generate efficient executable code. For example, a designer interested in efficient executable code might focus on statically typed variables, because the data type of such a variable need not be checked at runtime. Consider the following Java code segment, which declares and initializes the variables `i` and `s` and then uses them in later computations.

```
int i = 10;
String s = "My information";

// Do something with i and s
```

Because the data types of these two variables are known at compile time, the compiler can guarantee that only integer and string operations will be performed on them. Thus, the runtime system need not pause to check the types of these values before executing the operations.

In contrast, the equivalent code in Python simply assigns values to typeless variables:

```
i = 10
s = "My information"

# Do something with i and s
```

The absence of a type specification at compile time forces the runtime system to check the type of a Python variable's value before executing any operations on it. This causes execution to proceed more slowly than it would if the language were statically typed.

As another example, the early dialects of FORTRAN supported static storage allocation only. This meant that the memory requirements for all data declarations and subroutine calls had to be known at compile time. The number of positions in an array had to be declared as a constant, and a subroutine could not be recursive (a nonrecursive subroutine needs only one block of memory or activation record for its single activation, whereas a recursive routine requires potentially many activation records, whose number will vary with the number of recursive calls). This restriction allowed the memory for the program to be formatted just once, at load time, thus saving processing time as well as memory. In contrast, most modern languages require dynamic memory allocation at runtime, both for recursive subroutines and for arrays whose sizes cannot be determined until runtime. This support mechanism, whether it takes the form of a system stack or a system heap (to be discussed in Chapters 7 and 10), can incur substantial costs in memory and processing time.

Another view of efficiency is **programmer efficiency**: How quickly and easily can a person read and write a program in a particular language? A programmer's efficiency is greatly affected by a language's **expressiveness**: How easy is it to express complex processes and structures? Or, to put it another way: How easily can the design in the programmer's head be mapped to actual program code? This is clearly related to the language's abstraction mechanisms. The structured control statements of Algol and its successors are wonderful examples of this kind of expressiveness. If the programmer can describe making a choice or repeating a series of steps in plain English, the translation (by hand) of this thought process to the appropriate `if` statement or `while` loop is almost automatic.

The conciseness of the syntax also contributes to a language's programming efficiency. Languages that require a complex syntax are often considered less efficient in this regard. For designers especially concerned with programmer efficiency, Python is an ideal language. Its syntax is extremely concise. For example, unlike most languages, which use statement terminators such as the semicolon and block delimiters such as the curly braces in control statements, Python uses just indentation and the colon. Figure 2.1 shows equivalent multiway `if` statements in Python and C to illustrate this difference.

C	Python
<code>if (x > 0){</code>	<code>if x > 0.0:</code>
<code>numSolns = 2;</code>	<code>numSolns = 2</code>
<code>r1 = sqrt (x);</code>	<code>r1 = sqrt(x)</code>
<code>r2 = - r1;</code>	<code>r2 = -r1</code>
<code>}</code>	<code>elif x = 0.0:</code>
<code>else if (x == 0){</code>	<code>numSolns = 1</code>
<code>numSolns = 1;</code>	<code>r1 = 0.0</code>
<code>r1 = 0.0;</code>	<code>else:</code>
<code>}</code>	<code>numSolns = 0</code>
<code>else</code>	
<code>numSolns = 0;</code>	

Figure 2.1 Comparing the syntax of multiway `if` statements in C and Python

The absence of explicit data types in variable declarations in some languages allows for more concise code, and the support for recursion and dynamic data structures in most languages provides an extra layer of abstraction between the programmer and the machine. Of course, an exclusive focus on programmer efficiency can compromise other language principles, such as efficiency of execution and reliability.

Indeed, reliability can be viewed as an efficiency issue itself. A program that is not reliable can incur many extra costs—modifications required to isolate or remove the erroneous behavior, extra testing time, plus the time required to correct the effects of the erroneous behavior. If the program is unreliable, it may even result in a complete waste of the development and coding time. This kind of inefficiency is a resource consumption issue in software engineering. In this sense, programmer efficiency also depends on the ease with which errors can be found and corrected and new features added. Viewed in this way, the ease of initially writing code is a less important part of efficiency. Software engineers estimate that 90% of their time is spent on debugging and maintenance, and only 10% on the original coding of a program. Thus, maintainability may ultimately be the most important index of programming language efficiency.

Among the features of a programming language that help to make programs readable and maintainable, probably the most important is the concept of regularity. We turn to this in the next section.

2.3 Regularity

Regularity is a somewhat ill-defined quality. Generally, however, it refers to how well the features of a language are integrated. Greater regularity implies fewer unusual restrictions on the use of particular constructs, fewer strange interactions between constructs, and fewer surprises in general in the way language features behave. Programmers usually take the regularity of a language for granted, until some feature causes a program to behave in a manner that astonishes them. For this reason, languages that satisfy the criterion of regularity are said to adhere to a principle of least astonishment.

Often regularity is subdivided into three concepts that are more well-defined: generality, orthogonal design, and uniformity. A language achieves **generality** by avoiding special cases in the availability or use of constructs and by combining closely related constructs into a single more general one.

Orthogonal is a term borrowed from mathematics, where it refers to lines that are perpendicular. More generally, it is sometimes used to refer to two things that travel in independent directions, or that

function in completely separate spheres. In computer science, an **orthogonal design** allows language constructs to be combined in any meaningful way, with no unexpected restrictions or behavior arising as a result of the interaction of the constructs, or the context of use. The term **uniformity** refers to a design in which similar things look similar and have similar meanings and, inversely, in which different things look different.

These dimensions of regularity are explained with examples in the following sections. As you will see, the distinctions between the three are sometimes more a matter of viewpoint than actual substance, and you can always classify a feature or construct as irregular if it lacks just one of these qualities.

2.3.1 Generality

A language with generality avoids special cases wherever possible. The following examples illustrate this point.

Procedures and functions Function and procedure definitions can be nested in other function and procedure definitions in Pascal. They can also be passed as parameters to other functions and procedures. However, Pascal functions and procedures cannot be assigned to variables or stored in data structures. In C, one can create pointers to functions to treat them as data, but one cannot nest function definitions. By contrast, Python and most functional languages, such as Scheme and Haskell, have a completely general way of treating functions (nested functions definitions and functions as first-class data objects).

Operators In C, two structures or arrays cannot be directly compared using the equality operator `==`, but must be compared element by element. Thus, the equality operator lacks generality. This restriction has been removed in Smalltalk, Ada, Python, and (partially) in C++ and Java. More generally, many languages have no facility for extending the use of predefined operators (like `==` or `+`) to new data types. Some languages, however, such as Haskell, permit overloading of existing operators and allow new operators to be created by the user. In such languages, operators can be viewed as having achieved complete generality.

Constants In Pascal, the values assigned to constants may not be computed by expressions, while in Modula-2, these computing expressions may not include function calls. Ada, however, has a completely general constant declaration facility.

2.3.2 Orthogonality

In a language that is truly orthogonal, language constructs do not behave differently in different contexts. Thus, restrictions that are context dependent are considered nonorthogonal, while restrictions that apply regardless of context exhibit a mere lack of generality. Here are some examples of lack of orthogonality:

Function return types In Pascal, functions can return only scalar or pointer types as values. In C and C++, values of all data types, except array types, can be returned from a function (indeed, arrays are treated in C and C++ differently from all other types). In Ada, Python, and most functional languages, this lack of orthogonality is removed.

Placement of variable declarations In C, local variables can only be defined at the beginning of a block (compound statement),¹ while in C++ variable definitions can occur at any point inside a block (but, of course, before any uses).

Primitive and reference types In Java, values of scalar types (char, int, float, and so forth) are not objects, but all other values are objects. Scalar types are called **primitive types**, whereas object types are called **reference types**. Primitive types use **value semantics**, meaning that a value is copied during assignment. Reference types use **reference semantics**, meaning that assignment produces two references to the same object. Also, in Java, collections of objects are treated differently from collections of primitive types. Among Java's collections, only arrays can contain primitive values. Primitive values must be placed in special wrapper objects before insertion into other types of collections. In Smalltalk and Python, by contrast, all values are objects and all types are reference types. Thus, these languages use reference semantics only, and collections of objects are treated in an orthogonal manner.

Orthogonality was a major design goal of Algol68, and it remains the best example of a language where constructs can be combined in all meaningful ways.

2.3.3 Uniformity

The term **uniformity** refers to the consistency of appearance and behavior of language constructs. A language is uniform when similar things look similar or behave in a similar manner, but lacks uniformity when dissimilar things actually look similar or behave similarly when they should not. Here are some examples of the lack of uniformity.

The extra semicolon In C++, a semicolon is necessary after a class definition, but forbidden after a function definition:

```
class A { ... } ; // semicolon required
int f () { ... } // semicolon forbidden
```

The reason the semicolon must appear after a class definition is that the programmer can include a list of variable names before the semicolon, if so desired, as is the case with struct declarations in C.

Using assignment to return a value In Pascal, return statements in functions look like assignments to variables. This is a case where different things should look different, but instead look confusingly alike. For example:

```
function f : boolean;
begin
  ...
  f := true;
end;
```

Most other languages use a dedicated return statement for returning values from functions.

¹The ISO C Standard (ISO 9899 [1999]) removed this restriction.

2.3.4 Causes of Irregularities

Why do languages exhibit such irregularities at all? Surely the language designers did not intentionally set out to create strange restrictions, interactions, and confusions. Indeed, many irregularities are case studies in the difficulties of language design.

Take, for example, the problem with the semicolon in the C++ class declaration, noted as a lack of uniformity above. Since the designers of C++ attempted to deviate from C as little as possible, this irregularity was an essential byproduct of the need to be compatible with C. The lack of generality of functions in C and Pascal was similarly unavoidable, since both of these languages opt for a simple stack-based runtime environment (as explained in Chapter 10). Without some restriction on functions, a more general environment would have been required, and that would in turn have compromised the simplicity and efficiency of implementations. The irregularity of primitive types and reference types in Java is also the result of the designer's concern with efficiency.

On the other hand, it is worth noting that too great a focus on a particular goal, such as generality or orthogonality, can itself be dangerous. A case in point is Algol68. While this language was largely successful in meeting the goals of generality and orthogonality, many language designers believe that these qualities led to an unnecessarily obscure and complex language.

So how do you tell if a lack of regularity is reasonable? Consider the basic design goals of the language, and then think about how they would be compromised if the irregularity were removed. If you cannot justify a lack of regularity in this way, then it is probably a design flaw.

2.4 Security

As we have seen, the reliability of a programming language can be seriously affected if restrictions are not imposed on certain features. In Pascal, for instance, pointers are specifically restricted to reduce security problems, while in C they are much less restricted and, thus, much more prone to misuse and error. In Java, by contrast, pointers have been eliminated altogether (they are implicit in all object allocation), but at some cost of a more complicated runtime environment.

Security is closely related to reliability. A language designed with an eye toward security both discourages programming errors and allows errors to be discovered and reported. A concern for security led language designers to introduce types, type checking, and variable declarations into programming languages. The idea was to “maximize the number of errors that could not be made” by the programmer (Hoare [1981]).

However, an exclusive focus on security can compromise both the expressiveness and conciseness of a language, and typically forces the programmer to laboriously specify as many things as possible in the actual code. For example, LISP and Python programmers often believe that static type-checking and variable declarations make it difficult to program complex operations or provide generic utilities that work for a wide variety of data. On the other hand, in industrial, commercial, and defense applications, there is often a need to provide additional language features for security. Perhaps the real issue is how to design languages that are secure and that also allow for maximum expressiveness and generality. Examples of an advance in this direction are the languages ML and Haskell, which are functional in approach, allow multityped objects, do not require declarations, and yet perform static type-checking.

Strong typing, whether static or dynamic, is only one component of safety. Several modern languages, such as Python, Lisp, and Java, go well beyond this and are considered to be **semantically safe**.

That is, these languages prevent a programmer from compiling or executing any statements or expressions that violate the definition of the language. By contrast, languages such as C and C++ are not semantically safe. For example, an array index out of bounds error causes either a compile-time error or a runtime error in Python, Lisp, and Java, whereas the same error can go undetected in C and C++. Likewise, in languages such as C and C++, the programmer's failure to recycle dynamic storage can cause memory leaks, whereas in Python, Lisp, and Java, automatic garbage collection prevents this type of situation from ever occurring.

2.5 Extensibility

An **extensible** language allows the user to add features to it. Most languages are extensible at least to some degree. For example, almost all languages allow the programmer to define new data types and new operations (functions or procedures). Some languages allow the programmer to include these new resources in a larger program unit, such as a package or module. The new data types and operations have the same syntax and behavior as the built-in ones.

The designers of modern languages such as Java and Python also extend the built-in features of the language by providing new releases on a regular basis. These language extensions typically are backward-compatible with programs written in earlier versions of the language, so as not to disturb legacy code; occasionally, however, some earlier features become “deprecated,” meaning that they may not be supported in future releases of the language.

Very few languages allow the programmer to add new syntax and semantics to the language itself. One example is LISP (see Graham [2002]). Not only can Lisp programmers add new functions, classes, data types, and packages to the base language, they can also extend Lisp's syntax and semantics via a macro facility. A **macro** specifies the syntax of a piece of code that expands to other, standard Lisp code when the interpreter or compiler encounters the first piece of code in a program. The expanded code may then be evaluated according to new semantic rules. For example, Common Lisp includes a quite general `do` loop for iteration but no simple `while` loop. That's no problem for the Lisp programmer, who simply defines a macro that tells the interpreter what to do when it encounters a `while` loop in a program. Specifically, the interpreter or compiler uses the macro to generate the code for an equivalent `do` loop wherever a `while` loop occurs in a program. Figure 2.2 shows LISP code segments for a `do` loop and an equivalent `while` loop that compute the greatest common divisor of two integers `a` and `b`.

```
(do ()
  ((= 0 b))
  (let ((temp b))
    (setf b (mod a b))
    (setf a temp)))

(while (> b 0)
  (let ((temp b))
    (setf b (mod a b))
    (setf a temp)))
```

Figure 2.2 A Lisp `do` loop and a Lisp `while` loop for computing the greatest common divisor of `a` and `b`

Most readers will understand what the `while` loop in Figure 2.2 does without knowing Lisp, for it is very similar in structure to the `while` loop of C++, Java, or Python. However, the `do` loop is more complicated. The empty parentheses on the first line omit the variable initializers and updates that would normally go there. The Boolean expression on the second line denotes a termination condition rather

than a continuation condition. The rest of the code is the body of the loop. The `while` loop in Figure 2.3 would clearly be a simpler and better fit for this problem, but it does not already exist in Lisp.

Figure 2.3 shows the code for a macro definition that would enable Lisp to translate a `while` loop to an equivalent `do` loop.

```
(defmacro while (condition &rest body)
  '(do ()
      ((not ,condition))
      ,@body))
```

Figure 2.3 A Lisp macro that defines how to translate a Lisp `while` loop to a Lisp `do` loop

This code extends the syntax of the language by adding a new keyword, `while`, to it and telling the interpreter/compiler what to do with the rest of the code following the keyword in the expression. In this case, an expression that includes the keyword `do` and its appropriate remaining elements is substituted for the `while` loop's code during interpretation or compilation. LISP, thus, allows programmers to become designers of their own little languages when the syntax and semantics of the base language do not quite suit their purposes. Graham [2004] and Seibel [2005] provide excellent discussions of this powerful capability.

We now conclude this chapter with two case studies, where we briefly examine the design goals and criteria of two language developers, Bjarne Stroustrup [1994; 1996] and Guido van Rossum [2003]. They are primarily responsible for the popular languages C++ and Python, respectively.

2.6 C++: An Object-Oriented Extension of C

Bjarne Stroustrup's interest in designing a new programming language came from his experience as a graduate student at Cambridge University, England, in the 1970s. His research focused on a simulator for software running on a distributed system (a simulator is a program that models the behavior of a real-world object; in this case, the simulator pretended to be an actual set of distributed computers). Stroustrup first wrote this simulator in Simula67, because he found its abstraction mechanisms (primarily the class construct) to be ideal for expressing the conceptual organization of his particular simulator. In addition, he found the strong type-checking of Simula to be of considerable help in correcting conceptual flaws in the program design. He also found that the larger his program grew, the more helpful these features became.

Unfortunately, he also found that his large program was hard to compile and link in a reasonable time, and so slow as to be virtually impossible to run. The compilation and linking problems were annoying, but the runtime problems were catastrophic, since he was unable to obtain the runtime data necessary to complete his PhD thesis. After some study, he concluded that the runtime inefficiencies were an inherent part of Simula67, and that he would have to abandon that language in order to complete the project. He then rewrote his entire program in BCPL (a low-level predecessor of C), which was a language totally unsuited for the proper expression of the abstractions in his program. This effort was such a painful experience that he felt he should never again attempt such a project with the languages that were then in existence.

But, in 1979, as a new employee at Bell Labs in New Jersey, Stroustrup was faced with a similar task: simulating a UNIX kernel distributed over a local area network. Realizing that a new programming language was necessary, he decided to base it on C, with which he had become familiar at Bell Labs (considered the home of C and UNIX), and to add the class construct from Simula67 that he had found so useful.

Stroustrup chose C because of its flexibility, efficiency, availability, and portability (Stroustrup [1994], page 43). These qualities also fit nicely with the following design goals of the new language (ibid., page 23):

1. Support for good program development in the form of classes, inheritance, and strong type-checking
2. Efficient execution on the order of C or BCPL
3. Highly portable, easily implemented, and easily interfaced with other tools

2.6.1 C++: First Implementations

Stroustrup's first implementation of the new language came in 1979–80, in the form of a preprocessor, named Cpre, which transformed the code into ordinary C. The language itself, called **C with Classes**, added features that met most of the three goals listed above. Stroustrup called it a “medium success” (ibid., page 63). However, it did not include several important features, such as dynamic binding of methods (or virtual functions, as explained Chapters 5 and 9), type parameters (or templates, as explained in Chapters 5 and 9), or general overloading (as explained in Chapter 7). Stroustrup decided to expand the language in these and other directions, and to replace the preprocessor by a more sophisticated true compiler (which still generated C code as its target, for portability). The language that emerged in 1985 was called C++. Its compiler, **Cfront**, is still the basis for many compilers today. Stroustrup also expanded on the basic goals of the language development effort, to explicitly include the following goals:

1. C compatibility should be maintained as far as practical, but should not be an end in itself (“there would be no *gratuitous* incompatibilities,” ibid., page 101).
2. The language should undergo incremental development based firmly in practical experience. That is, C++ should undergo an evolution driven by real programming problems, not theoretical arguments, while at the same time avoiding “featurism” (adding cool features just because it is possible).
3. Any added feature must entail zero cost in runtime efficiency, or, at the very least, zero cost to programs that do not use the new feature (the “zero-overhead” rule, ibid., page 121).
4. The language should not force any one style of programming on a programmer; that is, C++ should be a “multiparadigm” language.
5. The language should maintain and strengthen its stronger type-checking (unlike C). (“No implicit violations of the static type system,” ibid., page 201.)
6. The language should be learnable in stages; that is, it should be possible to program in C++ using some of its features, without knowing anything about other, unused, features.
7. The language should maintain its compatibility with other systems and languages.

At this stage in its development, the language included dynamic binding, function and operator overloading, and improved type-checking, but not type parameters, exceptions, or multiple inheritance. These would all come later.

2.6.2 C++: Growth

In late 1985, Stroustrup published the first edition of his book on C++. In the meantime, Cpre and Cfront had already been distributed for educational purposes essentially at no cost, and interest in the language had begun to grow in industry. Thus, a first commercial implementation was made available in 1986. In 1987, the first conference specifically on C++ was organized by USENIX (the UNIX users' association). By October 1988, Stroustrup estimates that there were about 15,000 users of the language, with PC versions of C++ compilers appearing that same year. Between October 1979, when Stroustrup first began using Cpre himself and October 1991, Stroustrup estimates that the number of C++ users doubled every seven and a half months. The success of the language indicated to Stroustrup and others that a concerted attempt at creating a standard language definition was necessary, including possible extensions that had not yet been implemented.

2.6.3 C++: Standardization

Because C++ was rapidly growing in use, was continuing to evolve, and had a number of different implementations, the standardization presented a problem. Moreover, because of the growing user pool, there was some pressure to undertake and complete the effort as quickly as possible. At the same time, Stroustrup felt that the evolution of the language was not yet complete and should be given some more time.

Nevertheless, Stroustrup produced a reference manual in 1989 (the *Annotated Reference Manual*, or *ARM*; Ellis and Stroustrup [1990]) that included all of C++ up to that point, including proposals for new exception-handling and template (type parameter) mechanisms (multiple inheritance had already been added to the language and Cfront that same year). In 1990 and 1991, respectively, ANSI and ISO standards committees were convened; these soon combined their efforts and accepted the ARM as the "base document" for the standardization effort.

Then began a long process of clarifying the existing language, producing precise descriptions of its features, and adding features and mechanisms (mostly of a minor nature after the *ARM*) whose utility was agreed upon. In 1994, a major development was the addition of a standard library of containers and algorithms (the Standard Template Library, or STL). The standards committees produced two drafts of proposed standards in April 1995 and October 1996 before adopting a proposed standard in November 1997. This proposal became the actual ANSI/ISO standard in August 1998.

2.6.4 C++: Retrospective

Why was C++ such a success? First of all, it came on the scene just as interest in object-oriented techniques was exploding. Its straightforward syntax, based on C, was tied to no particular operating environment. The semantics of C++ incurred no performance penalty. These qualities allowed C++ to bring object-oriented features into the mainstream of computing. The popularity of C++ was also enhanced by its flexibility, its hybrid nature, and the willingness of its designer to extend its features based on practical experience. (Stroustrup lists Algol68, CLU, Ada, and ML as major influences after C and Simula67).

Is it possible to point to any “mistakes” in the design of C++? Many people consider C++ to be, like Ada and PL/I before it, a “kitchen-sink” language, with too many features and too many ways of doing similar things. Stroustrup justifies the size of C++ as follows (Stroustrup [1994], page 199):

The fundamental reason for the size of C++ is that it supports more than one way of writing programs. . . . This flexibility is naturally distasteful to people who believe that there is exactly one right way of doing things. . . . C++ is part of a trend towards greater language complexity to deal with the even greater complexity of the programming tasks attempted.

Stroustrup does, however, list one fact in the “biggest mistake” category (ibid., page 200)—namely, that C++ was first defined and implemented without any standard library (even without strings). This was, of course, corrected with the 1998 C++ standard. However, Java has shown that new languages must include an extensive library with graphics, user interfaces, networking, and concurrency. That is perhaps the biggest challenge for the C++ community in the future. Stroustrup himself comments (ibid., page 208): “In the future, I expect to see the major activity and progress shift from the language proper . . . to the tools, environments, libraries, applications, etc., that depend on it and build on it.”

2.7 Python: A General-Purpose Scripting Language

Guido van Rossum was part of a team at Centrum voor Wiskunde en Informatica (CWI) in The Netherlands that worked on the design and implementation of the programming language ABC in the mid-1980s. The target users of ABC were scientists and engineers who were not necessarily trained in programming or software development. In 1986, van Rossum brought his ideas from working on ABC to another project at CWI, called Amoeba. Amoeba was a distributed operating system in need of a scripting language. Naming this new language Python, van Rossum developed a translator and virtual machine for it. Another of his goals was to allow Python to act as a bridge between systems languages such as C and shell or scripting languages such as Perl. Consequently, he set out to make Python’s syntax simpler and cleaner than that of ABC and other scripting languages. He also planned to include a set of powerful built-in data types, such as strings, lists, tuples (a type of immutable list), and dictionaries. The dictionary, essentially a set of key/value pairs implemented via hashing, was modeled on a similar structure in Smalltalk but not present in other languages in the 1980s. The dictionary, which was also later implemented by Java’s map classes, proved especially useful for representing collections of objects organized by content or association rather than by position.

2.7.1 Python: Simplicity, Regularity, and Extensibility

Python is easy for nonprogrammers to learn and use in part because it is based on a small but powerful set of primitive operations and data types that can be easily extended. The novice first learns this small set of resources and then extends them by either defining new resources or importing resources from libraries defined by others who work in the relevant domain. However, unlike in some other languages, each new unit of abstraction, whether it is the function, the class, or the module, retains a simple, regular syntax. These features reduce the cognitive overload on the programmer, whether a novice or an expert. Guido van Rossum believed that lay people (that is, professionals who are nonprogrammers, but also students who just want to learn programming) should be able to get their programs up and

running quickly; a simple, regular syntax and a set of powerful data types and libraries has helped to achieve that design goal.

2.7.2 Python: Interactivity and Portability

A principal pragmatic consideration in the design of Python was the need to create a language for users who do not typically write large systems, but instead write short programs to experiment and try out their ideas. Guido van Rossum saw that a development cycle that provides immediate feedback with minimal overhead for input/output operations would be essential for these users. Thus, Python can be run in two modes: expressions or statements can be run in a Python shell for maximum interactivity, or they can be composed into longer scripts that are saved in files to be run from a terminal command prompt. The difference in interactivity here is just one of degree; in either case, a programmer can try out ideas very quickly and then add the results to an existing body of code. This experimental style in fact supports the iterative growth of reasonably large-scale systems, which can be developed, tested, and integrated in a bottom-up manner.

Another consideration in the design of Python was the diversity of the intended audience for the new language. The ideal language for this group of programmers would run on any hardware platform and support any application area. These goals are accomplished in two ways. First, the Python compiler translates source code to machine-independent byte code. A Python virtual machine (PVM) or runtime environment is provided for each major hardware platform currently in use, so the source code and the byte code can migrate to new platforms without changes. Second, application-specific libraries support programs that must access databases, networks, the Web, graphical user interfaces, and other resources and technologies. Over the years, many of these libraries have become integral parts of new Python releases, with new ones being added on a regular basis.

2.7.3 Python: Dynamic Typing vs. Finger Typing

In our discussion thus far, we have seen that Python appears to combine the elements of simple, regular syntax found in Lisp (although without the prefix, fully parenthesized expressions), Lisp's interactivity and extensibility, and the support of portability, object orientation, and modern application libraries found in Java. All of these features serve to bridge the gap, as van Rossum intended, between a systems language like C and a shell language like Perl. However, there is one other language element that, in van Rossum's opinion, makes the difference between fast, easy program development and slow, tedious coding. Most conventional languages, such as Ada, C, C++, and Java, are statically typed. In these languages, the programmer must specify (and actually type on the keyboard) the data type of each new variable, parameter, function return value, and component nested within a data structure. According to van Rossum, this restriction might have a place in developing safe, production-quality system software. In the context of writing short, experimental programs, however, static typing inevitably results in needless **finger typing** by the programmer. This excess work requirement, in turn, hampers programmer productivity. Instead of helping to realize the goal of saying more with less, static typing forces the programmer to say less with more.

Guido van Rossum's solution to this problem was to incorporate in Python the dynamic typing mechanism found in Lisp and Smalltalk. According to this scheme, all variables are untyped: any variable can name any thing, but all things or values have a type. The checking of their types does occur

but is deferred until runtime. In the case of Python, the PVM examines the types of the operands before running any primitive operation on them. If a type error is encountered, an exception is always raised. Thus, it is not correct to say that Python and other dynamically typed languages are weakly typed; the types of all values are checked at runtime.

The real benefit of deferring type checking until runtime is that there is less overhead for the programmer, who can get a code segment up and running much faster. The absence of manifest or explicit types in Python programs also helps to reduce their size; 20,000 lines of C++ code might be replaced by 5,000 lines of Python code. According to van Rossum, when you can say more with less, your code is not only easier to develop but also easier to maintain.

2.7.4 Python: Retrospective

Certainly van Rossum did not intend Python to replace languages such as C or C++ in developing large or time-critical systems. Because Python does not translate to native machine code and must be type-checked at runtime, it would not be suitable for time-critical applications. Likewise, the absence of static type checking can also be a liability in the testing and verification of a large software system. For example, the fact that no type errors happen to be caught during execution does not imply that they do not exist. Thorough testing of all possible branches of a program is required in order to verify that no type errors exist.

Python has penetrated into many of the application areas that Java took by storm in the late 1990s and in the first decade of the present century. Python's design goal of ease of use for a novice or nonprogrammer has largely been achieved. The language is now in widespread use, and van Rossum notes that it has made some inroads as an introductory teaching language in schools, colleges, and universities. The support for starting simple, getting immediate results, and extending solutions to any imaginable application area continues to be the major source of Python's appeal.

Exercises

- 2.1 Provide examples of one feature (in a language of your choice) that promotes and one feature that violates each of the following design principles:
 - Efficiency
 - Extensibility
 - Regularity
 - Security
- 2.2 Provide one example each of orthogonality, generality, and uniformity in the language of your choice.
- 2.3 Ada includes a `loop . . . exit` construct. PL/I also includes a similar `loop . . . break` construct. Is there a similar construct in Python or Java? Are these constructs examples of any of the design principles?
- 2.4 In Java, integers can be assigned to real variables, but not vice versa. What design principle does this violate? In C, this restriction does not exist. What design principle does this violate?
- 2.5 Choose a feature from a programming language of your choice that you think should be removed. Why should the feature be removed? What problems might arise as a result of the removal?

- 2.6 Choose a feature that you think should be added to a programming language of your choice. Why should the feature be added? What needs to be stated about the feature to specify completely its behavior and interaction with other features?
- 2.7 In Ada, the end reserved word must be qualified by the kind of block that it ends: `if . . . then . . . end if`, `loop . . . end loop`, and so on. In Python, begin/end blocks of code are marked by indentation. Discuss the effect these conventions have on readability, writability, and security.
- 2.8 Should a language require the declaration of variables? Languages such as Lisp and Python allow variable names to be used without declarations, while C, Java, and Ada require all variables to be declared. Discuss the requirement that variables should be declared from the point of view of readability, writability, efficiency, and security.
- 2.9 Languages such as Lisp and Python are dynamically typed, whereas languages such as C++ and Java are statically typed. Discuss the costs and benefits of each kind of typing.
- 2.10 The semicolon was used as an example of a nonuniformity in C++. Discuss the use of the semicolon in C. Is its use entirely uniform?
- 2.11 Two opposing views on comments in programs could be stated as follows:
- (a) A program should always contain elaborate comments to make it readable and maintainable.
 - (b) A program should be as much as possible self-documenting, with comments added sparingly only where the code itself might be unclear.

Discuss these two viewpoints from the point of view of language design. What design features might aid one viewpoint but not the other? What might aid both?

- 2.12 Here are two more opposing statements:
- (a) A program should never run with error checking turned off. Turning off error checking after a program has been tested is like throwing away the life preserver when graduating from the pool to the ocean.
 - (b) A program should always have error checking turned off after the testing stage. Keeping error checking on is like keeping training wheels on a bike after you've entered a race.

Discuss (a) and (b) from the point of view of language design.

- 2.13 Discuss the following two views of language design:
- (a) Language design is compiler construction.
 - (b) Language design is software engineering.
- 2.14 Two contrasting viewpoints on the declaration of comments in a programming language are represented by Ada and C: In Ada, comments begin with adjacent hyphens and end with the end of a line:

```
-- this is an Ada comment
```

In C, comments begin with `/*` and proceed to a matching `*/`:

```
/* this is a C comment */
```

Compare these two comment features with respect to readability, writability, and reliability. C++ added a comment convention (two forward slashes) similar to that of Ada. Why didn't C++ use exactly the Ada convention?

- 2.15 The principle of locality maintains that variable declarations should come as close as possible to their use in a program. What language design features promote or discourage this principle? How well do Python, C, C++, and Java promote this principle?
- 2.16 A language with a good symbolic runtime debugger is often easier to learn and use than a language that lacks this feature. Compare the language systems you know with regard to the existence and/or quality of a debugger. Can you think of any language constructs that would aid or discourage the use of a debugger?
- 2.17 Many human factors affect language use, from psychology to human-machine interaction (or ergonomics). For example, programming errors made by people are far from random. Ideally, a language's design should attempt to prevent the most common errors. How could most common errors be prevented by language design?
- 2.18 Most programming languages now use the free format pioneered by Algol60, in which statements can begin anywhere, but must end, not with the end of a line of text, but with an explicit end symbol, such as a semicolon. By contrast, Python and a few other languages use fixed format, in which statements begin in a particular column and end at the end of the line of code, unless continuation marks are provided. Discuss the effect of fixed or free format on readability, writability, and security.
- 2.19 Here is a quote from D. L. Parnas [1985]: "Because of the very large improvements in productivity that were noted when compiler languages were introduced, many continue to look for another improvement by introducing better languages. Better notation always helps, but we cannot expect new languages to provide the same magnitude of improvement that we got from the first introduction of such languages. . . . We should seek simplifications in programming languages, but we cannot expect that this will make a big difference." Discuss.
- 2.20 A possible additional language design principle is learnability—that is, the ability of programmers to learn to use the language quickly and effectively. Describe a situation in which learnability may be an important requirement for a programming language to have. Describe ways in which a language designer can improve the learnability of a language.
- 2.21 In most language implementations, the integer data type has a fixed size, which means that the maximum size of an integer is machine dependent. In some languages like Scheme, however, integers may be of any size, and so become machine independent. Discuss the advantages and disadvantages of making such "infinite-precision" integers a requirement of a language definition. Is it possible to also implement "infinite-precision" real numbers? Can real numbers be made machine independent? Discuss.
- 2.22 Fred Brooks [1996] lists five basic design principles:
1. Design, don't hack.
 2. Study other designs.
 3. Design top-down.
 4. Know the application area.
 5. Iterate.
- (a) Explain what each of these means in terms of programming language design.
 - (b) Describe to what extent the C++ design effort (Section 2.5) appears to meet each of the above design criteria.

2.23 Here is another quote from Stroustrup (Stroustrup [1994], page 45):

. . . language design is not just design from first principles, but an art that requires experience, experiments, and sound engineering tradeoffs. Adding a major feature or concept to a language should not be a leap of faith but a deliberate action based on experience and fitting into a framework of other features and ideas of how the resulting language can be used.

Compare this view of language design with the list of five principles in the previous exercise.

2.24 In Chapter 1, we discussed how the von Neumann architecture affected the design of programming languages. It is also possible for a programming language design to affect the design of machine architecture as well. Describe one or more examples of this.

Notes and References

Horowitz [1984] gives a list of programming language design criteria similar to the ones in this chapter, which he partially attributes to Barbara Liskov. For an extensive historical perspective on language design, see Wegner [1976] (this paper and the papers by Wirth [1974] and Hoare [1973] mentioned in this chapter are all reprinted in Horowitz [1987]). The rationale for the design of Ada is discussed in Ichbiah et al. [1979], where many design principles are discussed. Many design issues in modern languages are discussed in Bergin and Gibson [1996], especially the articles on Pascal (Wirth [1996]), C (Ritchie [1996]), C++ (Stroustrup [1996]), Lisp (Steele and Gabriel [1996]), CLU (Liskov [1996]), Algol68 (Lindsey [1996]), and Prolog (Colmerauer and Roussel [1996]). Another article in that collection (Brooks [1996]) gives general advice on language design. Stroustrup [1994] is a significantly expanded version of the C++ paper in that collection and gives a detailed analysis of design issues in C++. See also Hoare [1981] for some wry comments on designing languages, compilers, and computer systems in general. Gabriel [1996] and Graham [2004] contain provocative discussions of recent language design. The proceedings of the third ACM SIGPLAN conference on the history of programming languages (HOPL III) [2007] contain further material on recent language design.